Simulated annealing for 8 queens code:

```
import random
import math

N = 8
INITIAL_TEMPERATURE = 10.0
COOLING_RATE = 0.995
MIN_TEMPERATURE = 0.01
MAX_ITERATIONS = 5000


def cost(board):
    """Count all attacking pairs of queens."""
    conflicts = 0

    for i in range(N):
        for j in range(i + 1, N):
            same_row = board[i] == board[j]
            same_diagonal = (
                abs(board[i] - board[j]) == abs(i - j)
            )

            if same_row or same_diagonal:
                conflicts += 1

    return conflicts


def generate_neighbor(board):
    """Move one randomly chosen queen to a new row."""
    neighbor = board.copy()

    col = random.randrange(N)
    old_row = neighbor[col]

    possible_rows = [r for r in range(1, N + 1)
                     if r != old_row]

    neighbor[col] = random.choice(possible_rows)
    return neighbor


def print_grid(board):
    """Print the N-Queens solution as an N x N grid."""
    print("\nGrid Representation:")
    for row in range(1, N + 1):
        line = ["Q" if board[col] == row else "." for col in range(N)]
        print(" ".join(line))
```

```
def simulated_annealing():
    restart = 0

    while True:
        restart += 1
        current = [
            random.randint(1, N) for _ in range(N)
        ]
        current_cost = cost(current)
        temperature = INITIAL_TEMPERATURE

        print(f"\n========== Restart {restart} ==========")
        print("Initial state:", current)
        print("Initial cost:", current_cost)

        for step in range(1, MAX_ITERATIONS + 1):

            if current_cost == 0:
                print("\nGOAL FOUND!")
                print("Goal configuration:", current)
                print("Final cost:", current_cost)
                print("Steps in this restart:", step - 1)
                print_grid(current)
                return current

            if temperature < MIN_TEMPERATURE:
                break

            neighbor = generate_neighbor(current)
            neighbor_cost = cost(neighbor)

            delta = neighbor_cost - current_cost

            # Always accept improvements and equal-cost moves.
            if delta <= 0:
                probability = 1.0
                accept = True
            else:
                probability = math.exp(
                    -delta / temperature
                )
                accept = random.random() < probability

            print(f"\nStep {step}")
            print("Current state:", current)
            print("Current cost:", current_cost)
            print("Neighbor state:", neighbor)
```

```
            print("Neighbor cost:", neighbor_cost)
            print("Delta E:", delta)
            print(f"Temperature: {temperature:.4f}")
            print(f"Acceptance probability: {probability:.4f}")
            print("Decision:", "ACCEPT" if accept else "REJECT")

            if accept:
                current = neighbor
                current_cost = neighbor_cost

            temperature *= COOLING_RATE

        print("No solution in this run. Restarting...")


if __name__ == "__main__":
    simulated_annealing()
```

OUTPUT FOR SIMULATED ANNEALING:

```
Step 402
Current state: [7, 1, 8, 4, 2, 7, 3, 6]
Current cost: 2
Neighbor state: [5, 1, 8, 4, 2, 7, 3, 6]
Neighbor cost: 0
Delta E: -2
Temperature: 1.3398
Acceptance probability: 1.0000
Decision: ACCEPT

GOAL FOUND!
Goal configuration: [5, 1, 8, 4, 2, 7, 3, 6]
Final cost: 0
Steps in this restart: 402

Grid Representation:
. Q . . . . . .
. . . . Q . . .
. . . . . . Q .
. . . Q . . . .
Q . . . . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
PS D:\1BM24CS318\LAB-4>
```

HillClimbing code:

```
import random

N = 4

def cost(board):
    """Count pairs of queens attacking each other."""
    conflicts = 0
    for i in range(N):
        for j in range(i + 1, N):
            if (board[i] == board[j] or
                abs(board[i] - board[j]) == abs(i - j)):
                conflicts += 1
    return conflicts


def get_neighbors(board):
    """Move one queen to another row in its column (0-indexed: rows 0 to N-1)."""
    neighbors = []
    for col in range(N):
        for row in range(N):
            if row != board[col]:
                neighbor = board.copy()
                neighbor[col] = row
                neighbors.append((col, row, neighbor))
    return neighbors


def print_grid(board):
    """Print the N-Queens solution as an N x N grid."""
    print("\nGrid Representation:")
    for row in range(N):
        line = ["Q" if board[col] == row else "." for col in range(N)]
        print(" ".join(line))


def hill_climbing(initial_state=None):
    restart = 0

    while True:
        restart += 1
        # Use initial_state for the first attempt, then random boards for restarts
        if restart == 1 and initial_state is not None:
            current = initial_state.copy()
        else:
            current = [random.randint(0, N - 1) for _ in range(N)]

        print(f"\nRestart {restart}: Initial state = {current}")
        print("Initial cost =", cost(current))

        while True:
            current_cost = cost(current)

            if current_cost == 0:
                print("\nGOAL FOUND!")
                print("Final board:", current)
                print("Final cost:", current_cost)
                print_grid(current)
                return current

            neighbors = get_neighbors(current)

            print("\nCurrent state:", current)
            print("Current cost:", current_cost)
            print("Neighbor states and costs:")

            for col, row, neighbor in neighbors:
                print(
                    f" Move column {col} to row {row}: "
```

```
                    f"{neighbor}, cost = {cost(neighbor)}"
                )

            # Choose a minimum-cost neighbor.
            best_cost = min(cost(nb) for _, _, nb in neighbors)
            best_neighbors = [
                nb for _, _, nb in neighbors
                if cost(nb) == best_cost
            ]
            best = random.choice(best_neighbors)

            # Restart if no improvement is possible.
            if best_cost >= current_cost:
                print("Local minimum reached; restarting...")
                break

            current = best


if __name__ == "__main__":
    hill_climbing(initial_state=[3, 1, 2, 0])
```

HIllClimbing output:

```
Restart 5: Initial state = [1, 1, 0, 2]
Initial cost = 2

Current state: [1, 1, 0, 2]
Current cost: 2
Neighbor states and costs:
 Move column 0 to row 0: [0, 1, 0, 2], cost = 3
 Move column 0 to row 2: [2, 1, 0, 2], cost = 4
 Move column 0 to row 3: [3, 1, 0, 2], cost = 1
 Move column 1 to row 0: [1, 0, 0, 2], cost = 3
 Move column 1 to row 2: [1, 2, 0, 2], cost = 2
 Move column 1 to row 3: [1, 3, 0, 2], cost = 0
 Move column 2 to row 1: [1, 1, 1, 2], cost = 4
 Move column 2 to row 2: [1, 1, 2, 2], cost = 3
 Move column 2 to row 3: [1, 1, 3, 2], cost = 3
 Move column 3 to row 0: [1, 1, 0, 0], cost = 3
 Move column 3 to row 1: [1, 1, 0, 1], cost = 5
 Move column 3 to row 3: [1, 1, 0, 3], cost = 3

GOAL FOUND!
Final board: [1, 3, 0, 2]
Final cost: 0

Grid Representation:
. . Q .
Q . . .
. . . Q
. Q . .
```